

GNR638 - Kaggle Competition

Image Super-Resolution on a Custom Dataset Of Gaming Images

Anupam Rawat Rahul Choudhary
IIT Bombay IIT Bombay
22b3982@iitb.ac.in 200070065@iitb.ac.in

April 19, 2025

1 Introduction

Image super-resolution (SR) aims to reconstruct high-resolution (HR) images from their low-resolution (LR) counterparts, enhancing details critical for applications like gaming. We tried to get HR images from LR using two methods, evaluation of two Convolutional Neural Network (CNN)-based SR models: the baseline **Enhanced Deep Super-Resolution (EDSR)** model and an enhanced variant, **EDSR++**. We assess their performance on a custom dataset of gaming images, where each LR image has a corresponding HR ground truth. The goal is to upscale images by a factor of 4, preserving textures, edges, and visual fidelity. Performance is quantified using a joint metric combining **Peak Signal-to-Noise Ratio (PSNR)** and **Structural Similarity Index (SSIM)**.

2 Model Architectures

2.1 EDSR Model

The EDSR model is a deep CNN designed for efficient super-resolution. It follows a modular structure with three main components: **head**, **body**, and **upsample (tail)**.

2.1.1 Head Module

- **Purpose:** Maps the input LR image to a higher-dimensional feature space.
- **Configuration:** A single 3x3 convolutional layer with:
 - **Input Channels:** 3 (RGB).
 - **Output Channels:** 64 (feature maps).
 - **Padding:** 1 (to preserve spatial dimensions).
- **Mathematical Representation:**

$$F_0 = \text{Conv}_{3 \times 3}(I_{LR}, W_h, b_h)$$

where I_{LR} is the LR input, W_h and b_h are the weights and biases, and $F_0 \in \mathbb{R}^{64 \times H \times W}$.

2.1.2 Body Module

- **Purpose:** Extracts deep features and learns residual corrections.
- **Configuration:** 16 residual blocks, each with:

- Two 3x3 convolutional layers.
- **ReLU Activation:** Applied between convolutions ($\text{ReLU}(x) = \max(0, x)$).
- **Residual Connection:** Output added to the input.

- **Residual Block Details:**

- **First Conv:** $F_1 = \text{Conv}_{3 \times 3}(F_{in}, W_1, b_1)$.
- **Activation:** $F_2 = \text{ReLU}(F_1)$.
- **Second Conv:** $F_3 = \text{Conv}_{3 \times 3}(F_2, W_2, b_2)$.
- **Output:** $F_{out} = F_{in} + F_3$.

- **Why Residuals?:** Mitigates vanishing gradients, enabling deeper networks.

- **Total Body Operation:**

$$F_{body} = F_0 + \sum_{i=1}^{16} \text{ResBlock}_i(F_{i-1})$$

followed by a final 3x3 convolution to refine features.

2.1.3 Upsample (Tail) Module

- **Purpose:** Upscales features to HR dimensions.
- **Configuration:**
 - **Conv Layer:** Increases feature maps to $64 \times 16 = 1024$ (for scale factor 4, since $4^2 = 16$).
 - **PixelShuffle:** Rearranges features from $1024 \times H \times W$ to $64 \times 4H \times 4W$.
 - **Final Conv:** Maps back to 3 channels.
- **Mathematical Flow:**

$$F_{up} = \text{Conv}_{3 \times 3}(F_{body}, W_{up}, b_{up}), \quad F_{sh} = \text{PixelShuffle}(F_{up}), \quad I_{SR} = \text{Conv}_{3 \times 3}(F_{sh}, W_{out}, b_{out})$$

2.1.4 Code Implementation

```

1 import torch.nn as nn
2
3 class ResidualBlock(nn.Module):
4     def __init__(self, num_features):
5         super(ResidualBlock, self).__init__()
6         self.block = nn.Sequential(
7             nn.Conv2d(num_features, num_features, 3, padding=1),
8             nn.ReLU(inplace=True),
9             nn.Conv2d(num_features, num_features, 3, padding=1)
10        )
11    def forward(self, x):
12        return x + self.block(x)
13
14 class EDSR(nn.Module):
15    def __init__(self, scale_factor=4, num_blocks=16, num_features=64):
16        super(EDSR, self).__init__()
17        self.head = nn.Conv2d(3, num_features, 3, padding=1)
18        self.body = nn.Sequential(
19            *[ResidualBlock(num_features) for _ in range(num_blocks)],
20            nn.Conv2d(num_features, num_features, 3, padding=1)
21        )
22        self.upsample = nn.Sequential(
23            nn.Conv2d(num_features, num_features * (scale_factor ** 2), 3, padding=1),
24            nn.PixelShuffle(scale_factor),
25            nn.Conv2d(num_features, 3, 3, padding=1)

```

```

26         )
27     def forward(self, x):
28         x = self.head(x)
29         res = self.body(x)
30         x = x + res
31         return self.upsample(x)

```

2.2 EDSR++ Model

EDSR++ builds on EDSR with a deeper and wider architecture, incorporating a **res_scale** factor for stability.

2.2.1 Head Module

- **Configuration:** Maps 3 channels to 256 feature maps (vs. 64 in EDSR).
- **Purpose:** Increases model capacity for richer feature extraction.
- **Equation:**

$$F_0 = \text{Conv}_{3 \times 3}(I_{LR}, W_h, b_h)$$

with $F_0 \in \mathbb{R}^{256 \times H \times W}$.

2.2.2 Body Module

- **Configuration:** 32 residual blocks (double EDSR's depth), each with:
 - Two 3x3 convolutions.
 - ReLU between them.
 - **Res_scale Factor:** 0.1, applied to the block output before addition.
- **Residual Block Details:**
 - $F_{out} = F_{in} + 0.1 \times \text{Block}(F_{in})$.
- **Why Res_scale?:** Prevents gradient explosion in deep networks.
- **Body Equation:**

$$F_{body} = F_0 + \sum_{i=1}^{32} 0.1 \times \text{ResBlock}_i(F_{i-1})$$

2.2.3 Upsample (Tail) Module

- **Configuration:** Adjusted for 256 feature maps:
 - Conv to $256 \times 16 = 4096$ channels.
 - PixelShuffle to upscale.
 - Conv to 3 channels.
- **Flow:** Similar to EDSR but with larger feature dimensions.

2.2.4 Code Implementation

```
1 class ResidualBlock(nn.Module):
2     def __init__(self, channels, res_scale=0.1):
3         super(ResidualBlock, self).__init__()
4         self.res_scale = res_scale
5         self.block = nn.Sequential(
6             nn.Conv2d(channels, channels, 3, padding=1),
7             nn.ReLU(inplace=True),
8             nn.Conv2d(channels, channels, 3, padding=1)
9         )
10    def forward(self, x):
11        return x + self.block(x) * self.res_scale
12
13 class EDSRPP(nn.Module):
14    def __init__(self, scale_factor=4, num_blocks=32, num_feats=256):
15        super(EDSRPP, self).__init__()
16        self.head = nn.Conv2d(3, num_feats, 3, padding=1)
17        self.body = nn.Sequential(*[ResidualBlock(num_feats) for _ in range(num_blocks)])
18        self.tail = nn.Sequential(
19            nn.Conv2d(num_feats, num_feats * (scale_factor ** 2), 3, padding=1),
20            nn.PixelShuffle(scale_factor),
21            nn.Conv2d(num_feats, 3, 3, padding=1)
22        )
23    def forward(self, x):
24        x = self.head(x)
25        res = self.body(x)
26        x = x + res
27        return self.tail(x)
```

3 Dataset Preparation

3.1 EDSR Dataset

- **Class:** SRDataset.
- **Purpose:** Loads LR-HR image pairs.
- **Steps:**
 1. Read image paths from LR and HR directories.
 2. Convert images to RGB using OpenCV.
 3. Normalize to $[0,1]$ by dividing by 255.
 4. Transform to PyTorch tensors.

3.1.1 Code

```
:
1 import os
2 import cv2
3 from torch.utils.data import Dataset
4 from torchvision.transforms import ToTensor
5
6 class SRDataset(Dataset):
7     def __init__(self, lr_dir, hr_dir=None):
8         self.lr_files = sorted([os.path.join(lr_dir, f) for f in os.listdir(lr_dir)])
9         self.hr_files = sorted([os.path.join(hr_dir, f) for f in os.listdir(hr_dir)]) if
10         hr_dir else None
11     def __len__(self):
12         return len(self.lr_files)
13     def __getitem__(self, idx):
14         lr_img = cv2.imread(self.lr_files[idx])
15         lr_img = cv2.cvtColor(lr_img, cv2.COLOR_BGR2RGB).astype(np.float32) / 255.0
16         lr_tensor = ToTensor()(lr_img)
17         if self.hr_files:
18             hr_img = cv2.imread(self.hr_files[idx])
19             hr_img = cv2.cvtColor(hr_img, cv2.COLOR_BGR2RGB).astype(np.float32) / 255.0
20             hr_tensor = ToTensor()(hr_img)
21             return lr_tensor, hr_tensor
22         return lr_tensor, os.path.basename(self.lr_files[idx])
```

3.2 EDSR++ Dataset

- **Class:** PatchDataset.
- **Purpose:** Extracts patches to increase dataset size and reduce memory usage.
- **Details:**
 - **Patch Size:** 48x48 (LR), 192x192 (HR, since scale = 4).
 - **Stride:** 48 (controls overlap).
 - **Augmentation:** Random horizontal flipping (50% probability).
- **Steps:**
 1. Load full images.
 2. Extract patches using sliding windows.
 3. Randomly select one patch per image.
 4. Apply augmentation.

3.2.1 Code

```
:  
1 from PIL import Image  
2 import torch  
3 from torchvision import transforms  
4  
5 class PatchDataset(Dataset):  
6     def __init__(self, lr_dir, hr_dir, patch_size=48, stride=48):  
7         self.lr_paths = sorted([os.path.join(lr_dir, f) for f in os.listdir(lr_dir)])  
8         self.hr_paths = sorted([os.path.join(hr_dir, f) for f in os.listdir(hr_dir)])  
9         self.patch_size = patch_size  
10        self.stride = stride  
11        self.transform = transforms.ToTensor()  
12    def __len__(self):  
13        return len(self.lr_paths)  
14    def extract_patches(self, img, patch_size, stride):  
15        _, h, w = img.shape  
16        patches = []  
17        for i in range(0, h - patch_size + 1, stride):  
18            for j in range(0, w - patch_size + 1, stride):  
19                patch = img[:, i:i+patch_size, j:j+patch_size]  
20                patches.append(patch)  
21        return patches  
22    def __getitem__(self, idx):  
23        lr = self.transform(Image.open(self.lr_paths[idx]).convert('RGB'))  
24        hr = self.transform(Image.open(self.hr_paths[idx]).convert('RGB'))  
25        lr_patches = self.extract_patches(lr, self.patch_size, self.stride)  
26        hr_patches = self.extract_patches(hr, self.patch_size * 4, self.stride * 4)  
27        rand_idx = torch.randint(0, len(lr_patches), (1,)).item()  
28        lr_patch, hr_patch = lr_patches[rand_idx], hr_patches[rand_idx]  
29        if torch.rand(1).item() > 0.5:  
30            lr_patch = torch.flip(lr_patch, dims=[2])  
31            hr_patch = torch.flip(hr_patch, dims=[2])  
32        return lr_patch, hr_patch
```

4 Training Procedures

4.1 EDSR Training

- **Epochs:** 50.
- **Batch Size:** 4.
- **Loss Function: L1 Loss** (Mean Absolute Error):

$$L_1 = \frac{1}{N} \sum_{i=1}^N |I_{SR}(i) - I_{HR}(i)|$$

- **Pros:** Simple, focuses on pixel-wise accuracy.
- **Cons:** Sensitive to outliers.

- **Optimizer:** Adam ($\beta_1 = 0.9, \beta_2 = 0.999$), learning rate = $1e-4$.
- **Scheduler:** StepLR (reduce by 0.5 every 30 epochs).

4.1.1 Loss Curves

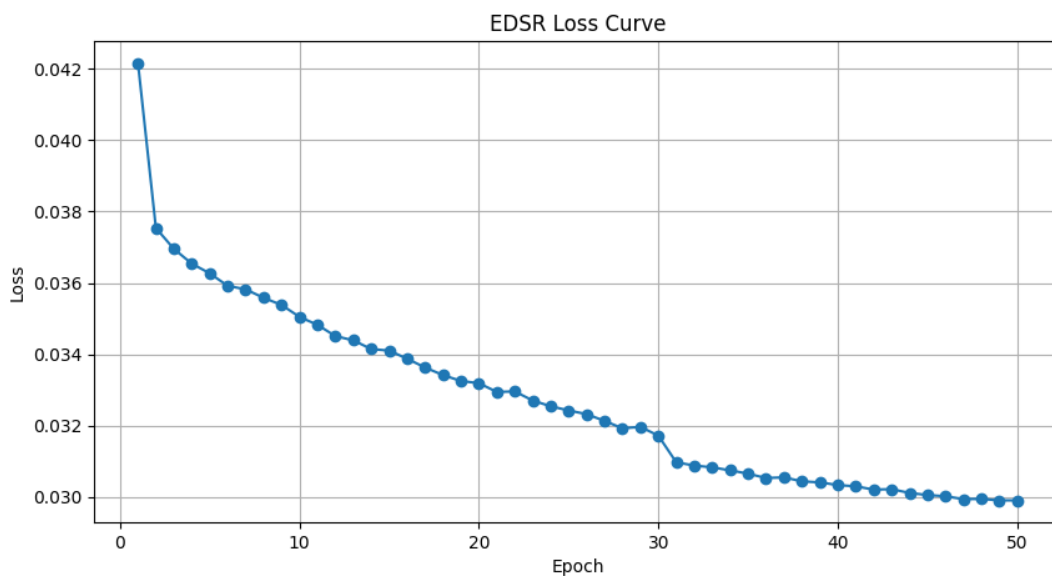


Figure 1: EDSR Loss Curves

4.1.2 Code

```
:  
1 import torch  
2 from torch.optim import Adam  
3 from torch.optim.lr_scheduler import StepLR  
4 from tqdm import tqdm  
5  
6 criterion = nn.L1Loss()  
7 optimizer = Adam(model.parameters(), lr=1e-4)
```

```

8 scheduler = StepLR(optimizer, step_size=30, gamma=0.5)
9
10 for epoch in range(50):
11     total_loss = 0
12     for lr_img, hr_img in tqdm(loader, desc=f'Epoch {epoch+1}/50'):
13         lr_img, hr_img = lr_img.to(device), hr_img.to(device)
14         sr = model(lr_img)
15         loss = criterion(sr, hr_img)
16         optimizer.zero_grad()
17         loss.backward()
18         optimizer.step()
19         total_loss += loss.item()
20     scheduler.step()
21     print(f"Epoch {epoch+1}, Loss: {total_loss / len(loader):.6f}")

```

4.2 EDSR++ Training

- Epochs: 200.
- Batch Size: 8.
- Loss Function: Charbonnier Loss:

$$L_{Char} = \sqrt{(I_{SR} - I_{HR})^2 + \epsilon^2}, \quad \epsilon = 1e-3$$

– **Advantages:** Robust to outliers, differentiable everywhere.

- **Optimizer:** Adam, learning rate = 1e-4.

4.2.1 Loss Curves

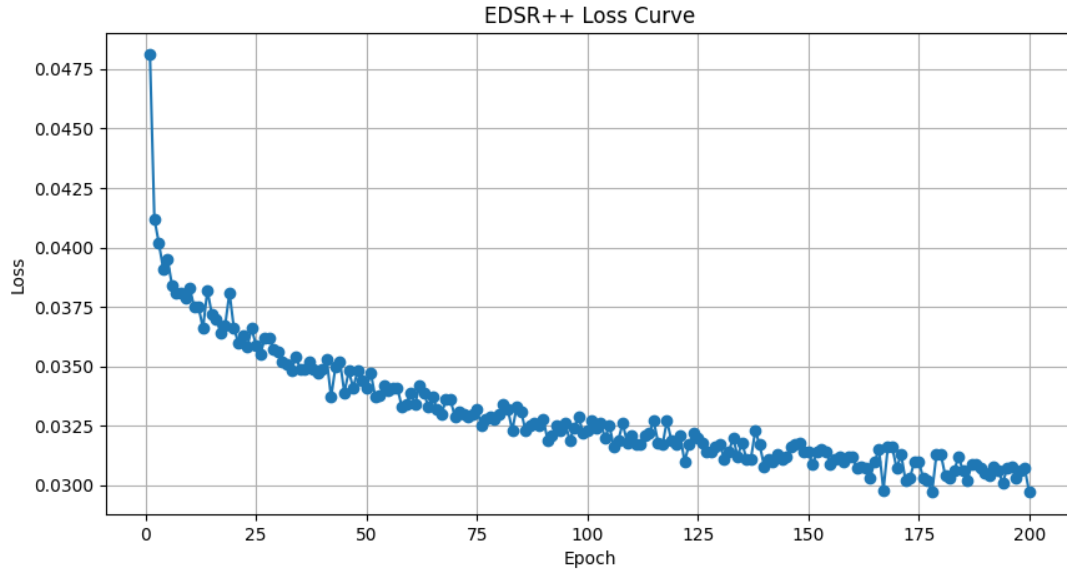


Figure 2: EDSR++ Loss Curves

4.2.2 Code

```
:
1 from torch.cuda.amp import GradScaler, autocast
2
3 class CharbonnierLoss(nn.Module):
4     def __init__(self, eps=1e-3):
5         super(CharbonnierLoss, self).__init__()
6         self.eps = eps
7     def forward(self, x, y):
8         diff = x - y
9         return torch.mean(torch.sqrt(diff * diff + self.eps ** 2))
10
11 criterion = CharbonnierLoss()
12 optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
13 scaler = GradScaler()
14
15 for epoch in range(200):
16     model.train()
17     epoch_loss = 0
18     for lr, hr in loader:
19         lr, hr = lr.to(device), hr.to(device)
20         optimizer.zero_grad()
21         with autocast():
22             sr = model(lr)
23             loss = criterion(sr, hr)
24             scaler.scale(loss).backward()
25             scaler.step(optimizer)
26             scaler.update()
27             epoch_loss += loss.item()
28     print(f"Epoch {epoch+1}/200, Loss: {epoch_loss / len(loader):.4f}")
```

5 Evaluation Metrics

- Performance is assessed with a **joint metric**:

$$\text{Joint Metric} = 40 \times \text{SSIM} + \text{PSNR}$$

5.1 PSNR

- **Definition:** Peak Signal-to-Noise Ratio.

- **Formula:**

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{N} \sum (I_{SR} - I_{HR})^2$$

– **MAX:** 255 (for 8-bit images).

- **Purpose:** Measures pixel-wise error.

5.2 SSIM

- **Definition:** Structural Similarity Index.

- **Formula:**

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

– μ : Mean, σ : Variance, c_1, c_2 : Stabilizing constants.

- **Range:** -1 to 1 (1 = perfect similarity).

- **Purpose:** Captures perceptual quality.

5.3 Joint Metric Rationale

- **Weighting:** SSIM is scaled by 40 to prioritize structural fidelity over raw pixel accuracy.

5.3.1 Code

:

```
1 from skimage.metrics import peak_signal_noise_ratio as psnr
2 from skimage.metrics import structural_similarity as ssim
3
4 def calculate_joint_metric(img1, img2):
5     psnr_value = psnr(img1, img2, data_range=255)
6     ssim_value, _ = ssim(img1, img2, full=True, multichannel=True)
7     return 40 * ssim_value + psnr_value
```

6 Inference

Inference generates SR images from LR inputs.

6.1 EDSR Inference

- Steps:

1. Load model and set to evaluation mode.
2. Process LR image.
3. Clamp output to [0,1].
4. Save as RGB image.

6.1.1 Code

```
:  
1 def inference(model, loader, output_dir='outputs/', device='cuda'):  
2     model.eval()  
3     os.makedirs(output_dir, exist_ok=True)  
4     with torch.no_grad():  
5         for lr_img, filename in tqdm(loader, desc='Generating images'):  
6             lr_img = lr_img.to(device)  
7             sr = model(lr_img).clamp(0, 1)  
8             sr_img = sr[0].cpu().permute(1, 2, 0).numpy() * 255  
9             sr_img = cv2.cvtColor(sr_img.astype(np.uint8), cv2.COLOR_RGB2BGR)  
10            cv2.imwrite(os.path.join(output_dir, filename[0]), sr_img)
```

6.2 EDSR++ Inference

Steps: Same as EDSR steps and includes multi-GPU support and AMP.

6.2.1 Code

```
:  
1 from torchvision.utils import save_image  
2  
3 def test(model_path, test_dir="lr/", output_dir="results"):  
4     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
5     model = EDSRPP().to(device)  
6     model = torch.nn.DataParallel(model)  
7     model.load_state_dict(torch.load(model_path))  
8     model.eval()  
9     os.makedirs(output_dir, exist_ok=True)  
10    transform = transforms.ToTensor()  
11    with torch.no_grad():  
12        for fname in sorted(os.listdir(test_dir)):  
13            lr_img = Image.open(os.path.join(test_dir, fname)).convert("RGB")  
14            lr_tensor = transform(lr_img).unsqueeze(0).to(device)  
15            with autocast():  
16                sr_tensor = model(lr_tensor)  
17            save_image(sr_tensor, os.path.join(output_dir, fname))
```

7 Conclusion

This experiment involved generating high-resolution (1920×1080) images from low-resolution (480×270) inputs using deep learning-based Super-Resolution models. The models were evaluated on a hold-out set of 500 LR images via PSNR (Peak Signal-to-Noise Ratio).

Table 1: Performance of Different Super-Resolution Models ($480 \times 270 \rightarrow 1920 \times 1080$)

Model Description	Epochs	Score (PSNR)
NLSA	100	57.9397
* EDSR++	200	58.5493
RCAN	50	57.5712
* EDSR	50	58.5604
SwinIR (pre-trained classical dataset)	-	54.7761
SRCNN with more epochs	100	56.6312
ViT	50	49.6894
SRCNN with RGB Channels	20	56.5044
SRCNN approach	20	50.8898

Below is a summary of the results of various models:

1. **Top Performers – EDSR Variants:**

EDSR with proper training (58.5604) and its improved version (58.5493) performed the best. This is expected since EDSR is optimized for performance on high-scale SR tasks, especially when sufficiently trained.

2. **RCAN NLSA:**

These attention-based models also showed competitive results (around 57.5–57.9), highlighting their ability to learn fine-grained features. NLSA leverages non-local self-attention effectively.

3. **SRCNN Its Variants:**

Although historically important, SRCNN has a simpler architecture and cannot capture complex patterns effectively for x4 scaling, explaining its relatively lower scores (50.9–56.6).

4. **SwinIR (Pre-trained for benchmarking) Performance Dip:**

SwinIR with pre-trained classical weights scored only 54.77, possibly due to a domain shift. Fine-tuning on the provided dataset could bridge this gap.

5. **ViT-Based Model:**

The Vision Transformer (ViT) performed poorly (49.6894), likely due to its high data and training time requirements, and lack of inductive biases like locality, which are crucial in SR tasks unless heavily optimized.

8 Code

All the code is on [NoviceCoderInfinity/GNR638_Machine_Learning_for_Remote_Sensing_II](#)